

Generative Chatbot

Farnoosh Javadi

Sept 17, 2023

Fine-tuning Strategy:

The finetuning is done with LORA based on the next word prediction task. During finetuning, model is seeing the whole conversation in the format of:

"#Farnoosh: Hello! #Hassan: Hello! How can I help you? #Farnoosh: I want to update my profile #Hassan: Sure!" and predicts the and predicts the next word. But in inference it sees the conversation until the final dialog and generates the final answer.

NOTE: since we have access to emotions as an additional step I also added emotion to each dialogue so that emotions are also considered during generation.

Next Steps: For inference I assumed that the emotion of final answer is given in the input (as the prompt), But the better approach was to first classify the dialogue emotion and then append the emotion to the conversation history for emotion-aware generation. But due to limited time I assumed we have the emotion prediction results.

Implementation Steps:

1) Data parsing and Dataset Creation

For parsing daily dialog dataset I changed the [original parser](#) in a way that it appends the speaker and the emotion to the dialogue too.

*Note that for the test split I considered the last dialogue as the label of the whole conversation as it is what we expect from the model to generate.

```
# Create Dictionary
d = {"text": [], "emotions": [], "acts": [], "text_label": []}

for line_count, (line_dial, line_emo, line_act) in enumerate(zip(in_dial, in_emo,
                                                                in_act)):
    emos = line_emo.split(' ')
    emos = emos[:-1]
    d['emotions'].append(emos)

    seqs = line_dial.split('__eou__')
    seqs = seqs[:-1]
    if mode == 'test':
        label = seqs.pop()
        seqs.append(" ")
    else:
        label = None

    d['text_label'].append(label)

    seqs = join_with_alterenate_prefixes(seqs, emos)
    d['text'].append(seqs)

    acts = line_act.split(' ')
    acts = acts[:-1]
    d['acts'].append(acts)

return d
```

Output:

Len(dataset_train) = 11118

Sample_data = "The kitchen stinks . __eou__ I'll throw out the garbage . __eou__"

Sample_data after parser = "#Farnoosh \$disgust: The kitchen stinks. #Hassan \$neutral: I'll throw out the garbage"

2) Model Loading

I used pretrained gpt2 from huggingface as my model since it has a reasonable number of params considering the small size of our fine-tuning dataset.

Note that I also tried Bloom-560m, and llama 7B too. For Bloom-560 the generation results were not as good as gpt2 as there were so much repetition. And as for llama-7B its finetuning taking so long so I decided to move forward with gpt2. Although the code is generalizable to any other AutoRegressive based model like gpt3, etc. and you can input the model_name in the command.

```
if mode=='train':

    #Load model
    model = AutoModelForCausalLM.from_pretrained(model_name)

    # Load tokenizer
    tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
    tokenizer.pad_token = tokenizer.eos_token
```

3) Creating finetuning config and trainer

For setting up the trainer there were some necessary steps:

1) defining LORA config

```
peft_config = LoraConfig(
    lora_alpha=lora_alpha,
    lora_dropout=lora_dropout,
    r=lora_r,
    bias="none",
    task_type="CAUSAL_LM"
)
```

2) defining trainer arguments

```
# set training args to be passed to trainer
training_arguments = TrainingArguments(
    output_dir=out_dir,
    per_device_train_batch_size=per_device_train_batch_size,
    gradient_accumulation_steps=gradient_accumulation_steps,
    optim=optim,
    save_steps=save_steps,
    logging_steps=logging_steps,
    learning_rate=learning_rate,
    fp16=True,
    max_grad_norm=max_grad_norm,
    num_train_epochs=num_train_epochs,
    warmup_ratio=warmup_ratio,
    group_by_length=True,
    lr_scheduler_type=lr_scheduler_type,
)
```

3) Setting up the trainer: I used SFTTrainer that gives a wrapper around transformers Trainer to easily fine-tune models on instruction based datasets using PEFT adapters.

```
trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    peft_config=peft_config,
    dataset_text_field="text",
    max_seq_length=max_seq_length,
    tokenizer=tokenizer,
    args=training_arguments,
)
```

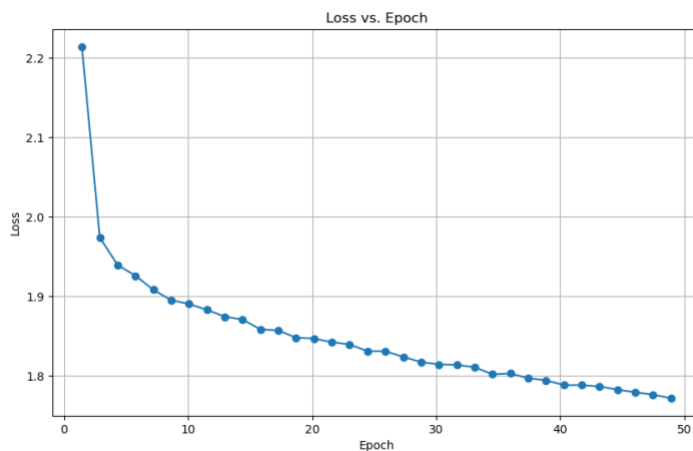
4) Train

I could simple use `trainer.train()`

```
trainer.train()
```

```
wandb: Run 'wandb online' or set WANDB_MODE=online to enable cl
0% 0/34750 [00:00<?, ?it/s]You're using a GPT2TokenizerFast t
{'loss': 2.214, 'learning_rate': 0.0001, 'epoch': 1.44}
{'loss': 1.974, 'learning_rate': 0.0001, 'epoch': 2.88}
{'loss': 1.9395, 'learning_rate': 0.0001, 'epoch': 4.32}
{'loss': 1.9261, 'learning_rate': 0.0001, 'epoch': 5.76}
13% 4630/34750 [16:57<1:51:35, 4.50it/s]
```

I wrote a script to plot `loss_curve`:



5) Evaluation

At first I needed to predict the next dialog per each data sample in training set:

```
for i, data in enumerate(dataset):
```

```
inp, label = data['text'], data['text_label']
input_ids = tokenizer(inp, return_tensors="pt").input_ids

pred_logits = model.generate(input_ids=input_ids, generation_config=GenerationConfig(max_new_tokens=20, min_
pred = tokenizer.decode(pred_logits[0], skip_special_tokens=False)

input_list = inp.split("#")
pred_list = pred.split("#")
answer = pred_list[len(input_list)-1]
answer = pred_list[len(input_list)-1][len(input_list[-1]):]
```

Sample Prediction: For more samples you can take a look at `log.txt`.

Input: "#Farnoosh \$neutral: Hey man , you wanna buy some weed ? #Hasan \$surprise: Some what ?
#Farnoosh \$neutral: Weed ! You know ? Pot , Ganja , Mary Jane some chronic ! #Hasan \$neutral: Oh ,
umm , no thanks . #Farnoosh \$neutral: I also have blow if you prefer to do a few lines . #Hasan \$neutral:
No , I am ok , really . #Farnoosh \$neutral: Come on man ! I even got dope and acid ! Try some ! #Hasan
\$neutral: Do you really have all of these drugs ? Where do you get them from ? #Farnoosh \$neutral: I got
my connections ! Just tell me what you want and I ' ll even give you one ounce for free . #Hasan \$neutral:
Sounds good ! Let ' s see , I want . #Farnoosh \$fear: Yeah ? #Hasan \$neutral:"

Answer: I am not sure what you are talking about?

Label: I want you to put your hands behind your head ! You are under arrest !

Then I did the evaluation based on rouge and bleu score.

```
def eval_metrics(model, dataset, tokenizer):

    answers, labels = inference(model, dataset, tokenizer)

    rouge = evaluate.load('rouge')
    res_rouge = rouge.compute(predictions=answers, references=labels)

    bleu = evaluate.load('bleu')
    res_bleu = bleu.compute(predictions=answers, references=labels)

    return res_rouge, res_bleu
```

Results:

Rouge: {'rouge1': 0.10297996132918898, 'rouge2': 0.023010147611108873, 'rougeL': 0.09625391605892789, 'rougeLsum': 0.09636512396510974}

Bleu: {'bleu': 0.012639909446031746, 'precisions': [0.2014664410603497, 0.02540766021994691, 0.009486847779215178, 0.0029930162953109413], 'brevity_penalty': 0.647357765983805, 'length_ratio': 0.6969339622641509, 'translation_length': 8865, 'reference_length': 12720}

```
.....
Rouge: {'rouge1': 0.10297996132918898, 'rouge2': 0.023010147611108873, 'rougeL': 0.09625391605892789, 'ro
Bleu {'bleu': 0.012639909446031746, 'precisions': [0.2014664410603497, 0.02540766021994691, 0.009486847779
```